

Security Assessment

Lanna Care for Animals shelter system: database, codebase and front end

Assessment date: 30 September 2026

Codebase reviewed: main branch at commit d26103b (release 0.10.0)

Stack: Next.js 16 on Cloudflare Workers, Supabase (Postgres, Auth, RLS), Google Drive

Prepared for: Lutan Bennett, developer and system owner

Prepared by: Claude (Anthropic), code review with manual verification of every finding cited

Contains details of security weaknesses. Share with the shelter's leadership and whoever will do the remediation; do not publish.

1. Executive summary

This report assesses the security of the Lanna Care for Animals system across its three layers: the Supabase database, the application code, and the front end including sign-in. The system's users are mostly non-technical shelter staff using phones in the field, so each recommendation is judged against that reality: a control that makes a keeper re-type a password in a kennel will be worked around, and a worked-around control is worse than none.

The overall picture is strong. For a small charity's custom application this is an unusually careful build. Row-level security is on every table; signed-out access is an explicit allow-list; role checks are repeated inside the database so the application cannot be bypassed; outside vet clinics are fenced to their own residents; role changes require authenticator-app verification; uploads are checked by their bytes; the service-role key never reaches a browser; secrets have never been committed; and production deploys are gated. Many of the decisions that led here are written down in the repository, which is itself a security asset.

No critical finding was identified. One finding is rated High: the weekly database backup, which contains password hashes, authenticator secrets and live session tokens, is uploaded unencrypted to a shared Google Drive folder. Thirteen findings are Medium. They fall into four groups: an open-redirect flaw in the sign-in return path; the absence of HTTP security headers; integrity gaps where records can be rewritten, silently “killed”, or have their files deleted despite the database refusing the operation; and privacy gaps where outside vets can read every carer's address and the public QR-card view lists every animal the shelter has ever had. The remaining findings are Low or informational and are mostly one-line hardening.

Nothing recommended here makes the app harder for field staff to use. The trade-offs that matter for them, session length, two-step scope, password rules and confirmation dialogs, are discussed in section 3 with a recommended setting for each. Several recommendations, such as plain-English error messages, better confirmation dialogs and an offline notice, improve day-to-day use as well as safety.

Overall posture at a glance

Area	Rating	Headline
Database	Good	Excellent access control; fix backup encryption, two integrity guards and vet/volunteer read scope.
Codebase	Good	Sound guard pattern; check row counts before Drive deletes, tidy secrets handling, small hardening items.
Front end & auth	Medium	Careful auth design; close the redirect bypass, add security headers, bound sign-ups, make admin two-step non-optional.

Findings by severity

Severity	Count	Meaning
Critical	0	Exploitable now by an outsider with serious consequences. None found.
High	1	Serious consequence if a plausible precondition occurs (for example a shared Drive folder being opened by the wrong person).
Medium	13	Real weakness needing a signed-in account, user interaction, or a mistake; fix in the next month.
Low	15	Defence in depth or narrow impact; fix opportunistically.

Severity	Count	Meaning
Info	4	A documented trade-off or observation to keep in view.

The five things to do first

1. **Encrypt the backups and exclude session tables** (DB-1), then rehearse a restore (DB-2). Half a day; removes the one High finding.
2. **Close the sign-in open redirect** (WEB-1) and **add security headers** (WEB-2). An hour of code; closes the two most exposed front-end weaknesses.
3. **Check the row count before deleting Drive files** (CODE-1, CODE-2) and **guard placement history** (DB-3, DB-4). A morning; protects medical records from silent damage.
4. **Bound sign-ups and paginate the user list** (WEB-3) and **prompt admins to enrol two-step** (WEB-4). Small changes that make the admin controls dependable.
5. **Narrow what vets and volunteers can read** (DB-5, DB-8) and **decide on the public roster** (DB-7). Personal-data decisions for the shelter's leadership, then one migration each.

2. Scope, method and limits

The assessment was a white-box review of the repository at commit d26103b on 30 September 2026. Three parallel reviews (database, server code, front end and authentication) each read their area in full; every finding cited in this report was then independently re-checked against the source by the lead reviewer, and the open-redirect bypass was reproduced with the URL parser. No live environment was attacked and no production data was read.

Reviewed

- All 116 migration files in `supabase/migrations/`, the migration runner, grant checker, public-view checker and backup scripts.
- All 56 server action and route handler files under `src/app/`, plus `src/lib/auth`, `supabase`, `uploads`, `google`, `status`, `assistant`, `links`, `csv`, `tags`, and the Cloudflare Worker wrapper and release-mail relay.
- The request proxy, public-path lists, sign-in, forgot-password, OAuth callback, password change, admin security pages, two-step verification, the public website pages, QR tag pages and all shared components.
- Configuration: `next.config.ts`, `wrangler.jsonc`, `.gitignore`, `.env.example`, CI workflow, `package.json`; git history for committed secrets; `npm audit` for production dependencies.
- Documentation: README, CLAUDE.md, the decision records and test plans that explain the security design.

Not reviewed (outside the code, or needs live access)

- Supabase dashboard settings: sign-up enabled or not, leaked-password protection, auth and MFA rate limits, redirect allow-list, session timeouts, whether TOTP is enabled on the production project. A checklist is in the appendix.
- Cloudflare zone settings: HSTS, WAF and rate-limiting rules, bot protection.
- The Google Cloud OAuth consent configuration and Drive folder sharing settings.
- The Raspberry Pi origin (`docs/pi-hosting.md`) and the developer laptop's disk encryption and account security.
- Dynamic testing against a running instance (penetration testing).

Severity scale

Severity	Definition
Critical	Remote, unauthenticated compromise of data or accounts. Stop and fix.
High	Serious loss of confidentiality or integrity under a realistic precondition. Fix within days.
Medium	Needs an account, a click, or a mistake; consequence is real. Fix within a month.
Low	Defence in depth, narrow impact, or insider-only. Fix when nearby.
Info	Deliberate trade-off or observation; no action required beyond awareness.

3. Security for people on phones in the field

The people using this system are keepers, volunteers and a few managers, working on phones, often outdoors, often with one hand. Security controls that assume a desk and a keyboard fail twice: they are bypassed, and the bypass leaves the system less safe than the control it replaced. The table below gives the recommended position on each control where security and ease of use pull in different directions, with the reasoning.

The guiding principle is: **make the rare, high-consequence actions (changing who has access, deleting records, restoring backups) deliberately slower, and keep the hundred-times-a-day actions (opening a resident, logging a weight, taking a photo) frictionless.** The current design already follows this principle in most places; the recommendations sharpen it.

Control	Recommended setting for this shelter	Why
Session length	No time-box; 60-day inactivity timeout; sign-out only by choice or when an admin archives the login.	A weekly user never re-enters a password. A lost phone signs out on its own within two months, and an admin can end it at once.
Two-step verification	Admins only, for Settings → Security only, once per sign-in, enrolment prompted at first sign-in.	The only action worth a second factor is changing who has access. Keepers, vets and volunteers never see it.
Password rules	Minimum 10 to 12 characters, breached-password check on, no complexity rules, password-manager attributes kept.	Length beats symbols on a phone keyboard. Breach checking stops the passwords attackers actually try.
Temporary passwords	Keep the current flow (generated, shown once, forced change on first sign-in).	Sending it over LINE is acceptable because it is single-use and expires the moment it is used.
Confirming deletions	The app's own dialog, naming the thing being deleted, large buttons, Cancel focused. Prefer archive over delete.	A native confirm box is tiny and reflexively tapped. Archive gives an undo; delete does not.
Error messages	Plain sentences (“That weight is already recorded for today”), never database wording.	Staff cannot act on a constraint name; a clear sentence prevents the double submission that follows confusion.
Offline	A visible “You're offline” banner; keep forms filled so a retry is one tap.	Kennels have poor signal. Silent failure looks like the app losing data.
Public QR cards	Shelter's decision: full roster public, or only adoptable animals to visitors with a sign-in prompt for the rest.	Staff scanning a tag are redirected to the hub either way; only visitors are affected.

Control	Recommended setting for this shelter	Why
Who reads contacts	Vets: name only. Volunteers: name and phone at most. Staff and above: everything.	Nobody outside the shelter needs a carer's home address, and volunteers rarely do.
Backups	Encrypted, automatic, from an always-on machine, rehearsed quarterly.	Invisible to users; the difference between a bad week and losing the shelter's records.

4. Database (Supabase / Postgres)

The database is the system's main line of defence: every table has row-level security (RLS) switched on, the anonymous (signed-out) role has been cut back to an explicit allow-list of public views, and the privileged functions re-check the caller's role inside the database rather than trusting the application. This is well above what most small charity systems have.

The findings below are therefore mostly about **integrity and privacy at the edges** (who can rewrite history, what an outside vet clinic can read, what the public QR cards reveal) and about the **backup file**, which is the one place the whole database, including sign-in secrets, exists outside Supabase's protection.

4.1 What is done well

- **RLS on every table, with no open write policy.** All 23 core tables enable RLS in `0001_initial_schema.sql` and every later table follows suit; no policy grants a write with `using (true)`.
- **Signed-out access is an allow-list, not a default.** Migration `0081` revokes everything from the anonymous role and grants back only the `public_*` views; `0082` does the same for functions and closes Postgres's default privileges so new objects start private.
- **Null-safe role guards.** After discovering that `NULL not in (...)` lets a role-less session through, every privileged function was rewritten to `is null or not in (0082)`. This is a subtle trap and the fix is documented so it will not recur.
- **Role changes require two-step verification at the database level.** Restrictive policies on `user_roles` (`0100`) refuse inserts, updates and deletes unless the session's JWT carries `aa12`, so even a stolen admin password cannot change who has access.
- **Outside vet clinics are fenced in.** Vet accounts see only residents their clinic has treated (`0108`) and write only for their own clinic (`0110`), enforced in the database, not just in the forms.
- **Public views are curated column by column.** Medical photos are excluded from the public gallery (`0101`, `0103`), the microchip number appears on no public view (`0113`), and Shelter Friend contact details are shown only with a per-field opt-in (`0076`).
- **Internal views were moved to a private schema** behind a `has_app_access()` gate (`0086`), so a signed-in account with no role cannot read them either.
- **Migration discipline.** Per-file transactions, a `schema_migrations` ledger, byte-for-byte comparison with `origin/main` before production applies, and a runtime script (`check-public-views.mjs`) that enumerates every exposed object and proves the anonymous role is refused everywhere but the allow-list.
- **No SQL injection path.** The dynamic SQL that exists is migration-time policy mirroring using `format('%I');`; no user input reaches `execute`.

4.2 Findings and mitigations

DB-1 Database backups are unencrypted and include sign-in secrets High

What we found. The weekly backup dumps both the `public` and `auth` schemas with `pg_dump` and uploads the file as-is to a `Backups/` folder in the shelter's Google Drive. The `auth` schema holds password hashes, authenticator-app (TOTP) secrets in plain text, and live refresh tokens. The shareable Drive link is also written to `backup.log`.

Evidence.

- `scripts/backup.mjs:44` sets `SCHEMAS = ["public", "auth"]`; lines 66-79 run `pg_dump --format=custom` with no encryption step.
- `scripts/backup.mjs:104` prints `Uploaded to Drive: <webViewLink>`; `backup.log` contains those links.
- README, Backups section: the folder sits under the same Drive root the app writes photos into.

Why it matters. Anyone who can open the shelter's Drive root, or who obtains the Drive OAuth token that the app itself holds, can download a complete copy of every resident, contact and medical record and, with the `auth` tables, reuse refresh tokens or attack password hashes offline. One file is the whole system.

Suggested mitigation.

- Encrypt the dump before upload with a key that lives only with the administrator (for example `age` or `gpg`, or `openssl enc -aes-256-cbc -pbkdf2`). Keep the key in a password manager, not in the repo or `.env` files.
- Exclude session material: add `--exclude-table=auth.refresh_tokens --exclude-table=auth.sessions --exclude-table='auth.mfa_*'` so a restore recreates accounts but everyone signs in afresh.
- Put backups in a Drive folder shared with the administrator only, not the shared application root; stop logging the web link.

Effect on field users. None. Backups are invisible to field staff.

Effort: Small (half a day) for encryption and exclusions; a restore rehearsal afterwards is essential.

DB-2 Backups depend on one laptop being awake, and a restore has never been rehearsed Medium

What we found. The scheduled task runs only when the developer's Windows account is logged on. The log shows only two runs, on 21 and 26 September, neither at the scheduled Sunday 03:00 slot, so both appear to be manual. Supabase's free tier has no point-in-time recovery, and the README notes that the restore procedure is untested.

Evidence.

- `scripts/backup-schedule.ps1:43-46` registers the task to run only while logged on.
- `backup.log` shows two runs: 2026-09-21T16:40Z and 2026-09-26T20:00Z.
- README, Backups: “Restoring ... untested until the backlog's restore rehearsal is done.”

Why it matters. The recovery point could be two weeks or more. A backup that has never been restored is a hope, not a plan: the first time it is needed is the worst time to discover a missing schema or a broken `pg_dump` version.

Suggested mitigation.

- Run the backup from something always on: a GitHub Actions scheduled workflow with the connection secret, or the Raspberry Pi the hosting plan already describes.
- Rehearse a restore into a scratch Supabase project once, write the steps down, and repeat quarterly.
- Make sure someone actually looks at the System status “Weekly backup” tile, or have the status alert mail admins when it goes red (the alert path already exists).

Effect on field users. None.

Effort: Small to medium (one day including the rehearsal).

DB-3 Placement history is not truly append-only: staff can rewrite end dates Medium

What we found. Staff and management have an unrestricted update policy on `placement_history`. The immutability trigger protects every column except `notes` and `end_date`, so a staff account can close a resident's active placement or reopen an old one by editing `end_date` directly through the data API.

Evidence.

- `supabase/migrations/0001_initial_schema.sql:529` — `staff_update_notes_placement_history` ... for update using (`current_user_role()` = 'staff') with no column restriction; mirrored for management in `0039`.
- `supabase/migrations/0049_undo_deceased.sql:45-64` — the trigger's error text itself says “immutable except notes and end_date (end_date is only closed automatically)”.

Why it matters. The lifecycle record the requirements call append-only can be rewritten without trace. A resident can be made to appear un-fostered or un-hospitalised, and the audit value of the table is lost.

Suggested mitigation.

- Extend `enforce_placement_history_immutability()` to raise when `end_date` changes unless a transaction-local flag set by `close_prior_placement()` is present (the same pattern the deceased lock already uses).
- Alternatively restrict the staff and management update policies with a `with check` that allows only `notes` to differ.

Effect on field users. None. The app never asks staff to edit an end date by hand.

Effort: Small (one migration).

DB-4 A plain enclosure move into a Lifecycle pseudo-enclosure can silently mark an animal deceased or adopted and lock its record Medium

What we found. A resident's status is derived from the name of the enclosure it sits in. Nothing stops a `ChangeEnclosure` placement (which volunteers can create) from targeting the Deceased or Adopted pseudo-enclosure. That path skips the proper deceased workflow, so no cascade snapshot is taken, every lock trigger engages, and the undo function refuses because it looks for a `Deceased` placement type.

Evidence.

- `0066` derives `current_status` from the enclosure name; `0026:124-141` `resident_is_deceased()` checks the enclosure too.
- `0001:572-573` volunteer insert policy constrains only `placement_type`; staff insert (`0001:528`) is unconstrained. No trigger references the Lifecycle zone on insert.
- `0049:165-173` `undo_deceased_placement()` requires `placement_type = 'Deceased'`.

Why it matters. One mis-tap on a phone (the enclosure picker lists real enclosures, but the data API does not care) or a malicious volunteer can “kill” or “adopt” any animal and leave a record only raw SQL can unlock.

Suggested mitigation.

- Add a `before insert` trigger on `placement_history` that raises when the target enclosure is in the Lifecycle zone (other than Unassigned) and the placement type does not match that pseudo-enclosure.
- Keep the picker as it is; this is a database guard, not a UI change.

Effect on field users. None; it only blocks a state the app never intends to create.

Effort: Small (one migration).

DB-5 Outside vet clinics and volunteers can read the whole contacts address book Medium

What we found. The `contacts` table holds carers' and donors' phone numbers, email, home address, LINE, WhatsApp and Messenger handles and free-text notes. Both the `vet` and `volunteer` roles have a blanket select policy on it, and migration `0105` records that this was deliberately not narrowed.

Evidence.

- `0001_initial_schema.sql:548 vet_read_contacts ... for select using (current_user_role() = 'vet') ;` line 569 the same for volunteers.
- `0105_vets_readonly_for_vets.sql:16` — “Deliberately not narrowed here: `vet_read_contacts ...`”.

Why it matters. A vet account belongs to an external business. With the anonymous key and its own session it can export every carer's home address and messaging handle in one request. That is personal data the shelter is responsible for under Thai PDPA and it is not needed for clinical work.

Suggested mitigation.

- Replace `vet_read_contacts` with a narrow view (`id, name, type`) or drop it; the resident hub only needs the carer's name.
- Consider the same for volunteers unless they genuinely phone carers; if they do, expose phone but not address and notes.

Effect on field users. Vets and volunteers lose nothing they use today. If a volunteer role does need a phone number, keep that column only.

Effort: Small (one migration plus a check that the vet's resident page still renders).

DB-6 No general audit trail for edits and deletes of resident, medical and contact records Medium

What we found. Most tables carry `created_by` but only four carry `updated_by`, and there is no before-image anywhere. The deceased workflow is the only event-sourced part of the system. Hard deletes are allowed on weights, attachments, maintenance jobs, medications and diet types.

Evidence.

- `updated_by` exists only on `site_content`, `site_pages`, `shelter_friends`, `fixed_outgoings`.
- `0001:532 staff_read_weight ... for all` (staff can delete weights); `0093:89-90` medication/diet deletes cascade their history away.

Why it matters. When a record is wrong there is no way to tell who changed it or what it said before, and a mistaken edit made on a phone in the field can only be recovered from a weekly backup. For medical data and personal data, accountability matters as much as access control.

Suggested mitigation.

- Add a generic `audit_log(table_name, row_id, op, actor, old_row, new_row, at)` table written by an `after insert/update/delete` trigger (security definer) on residents, contacts, prescriptions, vet_appointments, weight, attachments and immunization_records; readable by admins only.
- Prefer soft delete (an `archived_at` column, as contacts and users already have) over hard delete for medical rows.

Effect on field users. None visible. Later it enables a “recent changes” page and an undo for accidental edits, which is a real gain for field users.

Effort: Medium (one migration, half a day; an admin page to read it is optional).

DB-7 The public resident card and enclosure views publish the whole roster, and the card codes are guessable Medium

What we found. `public_resident_cards` (used by the QR/Rfid tag page `/r/<code>`) returns **every** resident, not only those marked public, including code, bio, temperament notes, intake date and a coarse status (Resident / Adopted / Deceased). Codes are sequential (R-0001, R-0002 ...) and the lookup is case-insensitive. `public_enclosures` (`/e/<id>`) then lists which residents live in each named enclosure or outreach site.

Evidence.

- `0103:...sql:132-168` — create or replace view `public_resident_cards` ... from residents `r` with no `is_public_visible` filter; granted to `anon`.
- `0022_resident_code_prefix.sql:10` sequential codes; `src/lib/residents/card.ts:55` case-insensitive lookup.
- `0079 public_enclosures` lists residents per enclosure; ids are UUIDs (not guessable) but a photographed QR is enough.

Why it matters. Anyone can walk `/r/R-0001` to `/r/R-9999` and build a complete inventory of every animal the shelter has ever had, including animals deliberately kept off the adoption site and deceased ones. No human personal data is exposed, so this is a privacy and reputational question rather than a safety one, and it may be exactly what the shelter wants for a tag that “must always answer”.

Suggested mitigation.

- Decision for the shelter: is a full public roster acceptable? If yes, record that decision and move on.
- If not: return only `is_public_visible` residents to signed-out callers and show “Sign in to see this animal” otherwise (staff are redirected to the hub anyway), and add a short random suffix to printed codes so they cannot be enumerated.

Effect on field users. For staff, nothing changes: a scanned tag still opens the resident. For visitors, non-public animals would show a sign-in prompt instead of a card.

Effort: Small (a view change plus a code-format change for newly printed tags).

DB-8 Every staff member's email is visible to vets and volunteers Low

What we found. The `app_users` view exposes every login's display name and email to any role that opens the app, including external vet clinics and volunteers.

Evidence.

- `0055:31-48`, `0063:48-66` define the view; `0086:142-143` gates it on any app role.

Why it matters. A staff directory with emails is a phishing starting point. Low, because the audience is small and semi-trusted.

Suggested mitigation.

- Exclude `email` from the view for the vet and volunteer roles (a `case when current_user_role() in ('admin', 'management', 'staff')` expression), or provide a separate admin-only view.

Effect on field users. None; the app shows display names, not emails, outside Settings.

Effort: Small.

DB-9 Staff can rename or delete the Lifecycle zone and its pseudo-enclosures that the status logic keys on Low

What we found. Status derivation, intake defaults and the deceased lock all look up enclosures by name (Deceased, Adopted, Hospital, Fostered, Unassigned), and staff have full read/write on `zones` and `enclosures`.

Evidence.

- `0001:522-523 staff_rw_zones, staff_rw_enclosures ... for all; 0026:138-139` and `0090:126` match on the names.

Why it matters. An accidental rename in Settings would break intake and status for every resident until reverted.

Suggested mitigation.

- Add a trigger that refuses rename or delete of the Lifecycle zone and its five enclosures, or mark them with a `protected` flag the admin UI respects.

Effect on field users. None.

Effort: Small.

DB-10 Static migration checks do not cover RLS, `search_path` or anon revokes on new objects Low

What we found. `check-migration-grants.mjs` verifies that new tables and views carry a grant and that gated views keep their `has_app_access()` clause. It does not check that a new table enables RLS, that a `security definer` function pins `search_path`, or that a new function revokes anon. Those are caught only at runtime by `check-public-views.mjs`. Separately, `site_content`, `site_pages` and `site_content_photos` are base tables selectable by anon with `select *`, so any column added later is world-readable by default.

Evidence.

- `scripts/check-migration-grants.mjs:146-191; 0018:50,54, 0059:45` public read policies on base tables.

Why it matters. A future migration written in a hurry could ship a table without RLS and only be noticed if someone runs the runtime check.

Suggested mitigation.

- Extend the static check: `create table requires enable row level security` in the same file; `security definer requires set search_path; forbid grant ... to anon` on objects not named `public_*` or `site_*`.
- Run `check-public-views.mjs` in CI against the test database, not only by hand.
- Expose the three `site_*` tables to anon through fixed-column views.

Effect on field users. None.

Effort: Small.

DB-11 Assistant transcripts are kept verbatim, forever [Info](#)

What we found. `assistant_actions.request_text` stores whatever staff typed into the assistant, readable by management. Staff may type names, phone numbers or health details.

Evidence.

- `0070:57-59`.

Why it matters. A slow accumulation of personal data with no stated retention.

Suggested mitigation.

- State a retention period (for example 90 days) and add a scheduled delete, or store only the parsed intent.

Effect on field users. None.

Effort: Small.

5. Codebase (server actions, API routes, Worker, scripts)

The application layer is Next.js 16 with Server Actions and a handful of Route Handlers, deployed to a Cloudflare Worker. Every request passes a proxy that refreshes the session and refuses signed-out or role-less users, then each action either checks the role explicitly or relies on the database's RLS and privileged functions to refuse it.

The review read all 56 action and route files. The pattern is sound. The findings are places where the JavaScript layer performs a side effect (usually deleting a Google Drive file) before it knows whether the database accepted the corresponding write, plus operational matters: where secrets live, error messages, exports and dependencies.

5.1 What is done well

- **The service-role key never reaches the browser.** Only three variables carry the `NEXT_PUBLIC_` prefix (URL, anonymous key, site URL). `createAdminClient()` is imported only by admin security, the user's own password change and the status checks, and every admin security action checks admin role and two-step before using it.
- **Cron and relay endpoints use a constant-time bearer compare** (`src/app/api/status/alerts/route.ts:16-24`, `worker/release-mail.mjs:44-50`), and the Worker's cron calls the handler in-process so the key never crosses the network.
- **Uploads are checked by their bytes, not their name.** A MIME allow-list plus magic-byte sniffing (`src/lib/uploads/file-signature.ts`), a 15 MB cap, SVG never accepted, role check before any Drive call, and an orphaned Drive file removed if the database insert fails.
- **The photo proxy asks the database who is asking.** Public files are served from the edge cache; private ones only if the caller's own RLS can see a row holding that file id, and they are served `private`, `no-store` so a blood test never lingers in a shared shelter PC's browser cache. Unknown and refused look identical (404).
- **Server-side request forgery is closed.** The only outbound fetches to user-influenced addresses are the map-link preview, which allows only Google's short-link hosts, follows at most three hops and gives up after three seconds.
- **External links are validated to https with host allow-lists** in code (`src/lib/links/validate.ts`) and again by a database check constraint.
- **Secrets hygiene in the pipeline.** `.env*` and `.dev.vars` are gitignored and were never committed (checked across all history); deploy pushes secrets through `wrangler secret bulk` on stdin; the OpenNext env snapshot is stripped so `.env.local` never ships; status output and alert mail run through `redactSecrets`.
- **Deploys are gated.** UAT and production refuse unless the checkout is a clean, pushed `main` with a written-down release; CI runs typecheck, lint and build with placeholder credentials and holds no real secrets.
- **The assistant is deterministic.** It is a regex parser, not a language model, so there is no prompt-injection surface; its writes go through the same role-checked helpers the forms use.
- **Actions return refusals rather than throwing**, so a phone user sees a sentence, not a framework error code.

5.2 Findings and mitigations

CODE-1 A Drive file is deleted even when the database quietly refused the delete Medium

What we found. Four attachment-delete actions run `.delete().eq('id', ...)` without asking how many rows were affected, then delete the Google Drive file. Under RLS a refused delete is not an error; it simply affects zero rows. A vet can select attachments for any resident in its scope but delete only its own clinic's, so the database keeps the row while the code removes the file.

Evidence.

- `src/app/residents/[id]/blood-tests/actions.ts:32-42` and `.../procedures/actions.ts:32-42`: delete with no `.select()`, then `getDriveClient().deleteFile(...)`.
- Same pattern in `src/app/maintenance/actions.ts:361-369` and `src/app/projects/actions.ts:322-329`.
- `0110_vet_own_clinic_writes.sql:186-235` — vets may select more attachments than they may delete.
- `src/lib/google/drive.ts:408-412` — Drive delete is permanent, not trash.

Why it matters. An external vet account can permanently destroy another clinic's blood-test or procedure document while the record still points at it. Medical record integrity is lost and the failure is silent.

Suggested mitigation.

- Use `.delete().eq(...).select('id')` and abort before touching Drive unless exactly one row came back.
- Better: route these through a security definer RPC like `delete_resident_photo`, which already does the role check inside the database, and move Drive files to trash rather than deleting permanently.

Effect on field users. None; the correct case behaves exactly as today.

Effort: Small (four files, an hour).

CODE-2 Deleting a maintenance job removes its attachment records before finding out the caller may not delete the job Medium

What we found. `deleteMaintenanceJob` deletes all attachment rows for the job first, then tries to delete the job, and only then checks whether zero rows came back and returns “not authorised”. Volunteers have full write on attachments but no delete on maintenance.

Evidence.

- `src/app/maintenance/actions.ts:308-323` — attachments deleted at 309-313, job delete and zero-row check at 315-323.
- `0001_initial_schema.sql:583` `volunteer_rw_attachments ... for all`; volunteers have no delete on maintenance.

Why it matters. Any volunteer can strip every photo record from any maintenance job by invoking the action directly, and the Drive files are orphaned.

Suggested mitigation.

- Delete the job first and let a foreign-key cascade remove the attachment rows, or check the role up front the way `deceased/actions.ts:92-95` does.
- Ideally wrap the whole delete in one transactional RPC.

Effect on field users. None.

Effort: Small.

CODE-3 All production secrets sit in plain-text files on one laptop and are copied into every worktree**Medium**

What we found. `.env.local` holds the service-role key, a Supabase personal access token with power over every project in the account, the Google OAuth client secret and refresh token, and a dev test user's password.

`.env.deploy.production` holds the production service-role key, the database password and the Google sign-in secret. The worktree script copies `.env.local` into each new feature folder, so the same secrets exist in several places at once.

Evidence.

- Variable names present in `.env.local`, `.env.deploy.production`, `.dev.vars` (values not inspected).
- CLAUDE.md, Workstreams: `worktree.mjs` new “copies `.env.local`”.
- `.gitignore` covers all of them and git history confirms none was ever committed.

Why it matters. Loss or compromise of that one laptop is loss of production, the Drive and the backups in one go, with no need to defeat two-step verification. Several copies widen the chance one is caught by a sync tool, a screenshot or an AI session transcript.

Suggested mitigation.

- Move the personal access token and database password into Windows Credential Manager or a password manager CLI and inject them at run time; the scripts already read from `process.env`.
- Use a scoped token for backups and deploys rather than the account-wide PAT; rotate the DB password and service-role key once the new arrangement is in place.
- Turn on BitLocker on the laptop if it is not already, and consider whether worktrees need the full file or just the dev database keys.

Effect on field users. None for shelter users.

Effort: Small to medium (half a day plus rotation).

CODE-4 CSV exports do not neutralise spreadsheet formulas**Low**

What we found. `csvField` quotes a field only when it contains a comma, quote or line break. A medication or diet name that starts with `=`, `+`, `-` or `@` is written as-is and Excel will evaluate it when the Stock usage or Cashflow CSV is opened.

Evidence.

- `src/lib/csv.ts:7-8`; names flow in from `src/app/management/stock-usage/page.tsx:369-384`.

Why it matters. Only management and admin can create those names, so this is an insider or mistake risk rather than an external one, but a formula can exfiltrate data or run a command from the spreadsheet.

Suggested mitigation.

- In `csvField`, prefix a value that begins with `=`, `+`, `-`, `@`, tab or carriage return with a single quote unless it parses as a plain number.

Effect on field users. None.

Effort: Trivial.

CODE-5 Raw database error text is shown to users in several forms**Low**

What we found. About a dozen actions return `error.message` from Postgres verbatim (constraint names, column names, trigger wording). Some actions already translate known error codes into plain sentences.

Evidence.

- `blood-tests/new/actions.ts:58`, `diets/actions.ts:104`, `immunizations/new/actions.ts:92`, `procedures/new/actions.ts:104`, `vet-visits/new/actions.ts:63`, `residents/new/actions.ts:130`; good examples in `prescriptions/actions.ts:36-41` and `src/lib/action-result.ts:28-36`.

Why it matters. Leaks schema detail to signed-in users (minor) and, more importantly for this audience, shows non-technical staff SQL wording they cannot act on.

Suggested mitigation.

- Map the known codes (unique violation, check violation, RLS refusal) to the existing translated messages and send everything else through `unexpectedFailure`, which logs it with a reference.

Effect on field users. Improves it: staff see “That reading already exists for today” rather than a constraint name.

Effort: Small.

CODE-6 Drive folder-name queries escape quotes but not backslashes**Low**

What we found. Names are escaped with `replace(/'/g, '\\')` before being placed in a Drive `q=` query. A name ending in a backslash escapes the closing quote and breaks or alters the query.

Evidence.

- `src/lib/google/drive.ts:473`, `:890`, `:953`.

Why it matters. An upload fails, or in a contrived case a photo is filed into the wrong folder inside the shelter's own Drive. No data exposure.

Suggested mitigation.

- Escape backslashes first: `name.replace(/\\/g, '\\\\').replace(/'/g, '\\')`.

Effect on field users. None.

Effort: Trivial.

CODE-7 PDF signature check is loose and the photo proxy sets no content-type protection**Low**

What we found. A file is accepted as PDF if `%PDF-` appears anywhere in its first kilobyte, so an HTML/PDF polyglot can be stored. The proxy serves Drive's content type with no `X-Content-Type-Options: nosniff` or `Content-Disposition`.

Evidence.

- `src/lib/uploads/file-signature.ts:74`; `src/app/api/photos/[fileId]/route.ts:126-134`.

Why it matters. Not exploitable today because the stored type is the app's own image or PDF type and SVG/HTML are never accepted; it is defence in depth against a future change.

Suggested mitigation.

- Require `%PDF-` as the first non-whitespace bytes; add `X-Content-Type-Options: nosniff` and `Content-Disposition: inline; filename=...` on the proxy response (see WEB-2 for headers generally).

Effect on field users. None.

Effort: Trivial.

CODE-8 No application-level rate limiting; a signed-in user could burn the Drive quota Low

What we found. Nothing in the app limits request rates. Private photos are deliberately never cached, so every view hits Google Drive; upload routes hit Drive per call. Sign-in throttling is Supabase's, not the app's.

Evidence.

- No limiter in any route or action; `api/photos/[fileId]/route.ts:132 private, no-store`.

Why it matters. A misbehaving script with a staff session, or a bug in a loop, can exhaust the Drive API quota and take photos down for everyone. Public traffic is already absorbed by the edge cache.

Suggested mitigation.

- Add a Cloudflare rate-limiting rule on `/api/*` and `/login` (free tier includes one rule); it needs no code and no user impact.

Effect on field users. None at sane thresholds (for example 100 requests per minute per IP).

Effort: Trivial (dashboard setting).

CODE-9 Dependency advisories in development tooling; no audit step in CI; actions not pinned Low

What we found. `npm audit --omit=dev` reports one high and two moderate advisories, all in the chain `@opennextjs/cloudflare → wrangler → miniflare → undici`. Undici is the local dev runtime's HTTP client and is not in the deployed Worker bundle. Runtime dependencies (Next 16.3.5, Supabase SSR 0.12.7, supabase-js 2.116) are current. CI does not run an audit, and GitHub Actions are referenced by tag rather than commit SHA.

Evidence.

- Audit output, 2026-09-30: `{ moderate: 2, high: 1, critical: 0 }`; `.github/workflows/ci.yml`.

Why it matters. Low today. Without an audit step the next genuinely exposed advisory in Next or Supabase would go unnoticed until someone runs it by hand.

Suggested mitigation.

- Bump `wrangler` (a fix is available); add `npm audit --omit=dev --audit-level=high` to CI as a non-blocking job first; pin actions to SHAs.

Effect on field users. None.

Effort: Trivial.

CODE-10 About twenty pages and the manual PDF route rely on the proxy alone for the sign-in redirect**Info**

What we found. Role-gated pages call `requireRole` and friends. Around twenty ordinary pages (blood tests, diets, prescriptions, weight, vet visits, assistant, manual) do not check the session themselves; they rely on the proxy for the redirect and on RLS for data. The manual PDF route renders the whole manual when no role is found.

Evidence.

- Grep of `src/app/**/page.tsx` for guard calls; `src/app/manual/pdf/route.ts:37-42`.

Why it matters. None today: a signed-out request that somehow skipped the proxy would render an empty page, because every query uses the cookie-scoped client. It becomes relevant only if the public-path list or the proxy matcher changes (see WEB-8).

Suggested mitigation.

- Add a one-line `getUser()` check to the manual PDF route; optionally a shared layout guard for the app section so new pages inherit it.

Effect on field users. None.

Effort: Small.

5.3 Authorisation guard by action and route

Every server action and route handler was mapped to the check that protects it. “RLS only” means the database's policies are the guard, which is legitimate in this design as long as the code checks the row count before any side effect (CODE-1, CODE-2).

Area	Guard
Settings pages (admin/*) actions	<code>hasAdminRole</code> first line; security actions also require two-step (<code>aa12</code>)
Management pages actions	<code>assertManagementRole</code> / <code>hasManagementRole</code> first line
Stocktake, deliveries, assistant writes	Explicit role allow-list plus RPC re-check
Resident edit, move, hospital, rehome, deceased, adoption updates, photos	Explicit role list or <code>security definer</code> RPC with role check
Blood tests, procedures, diets, immunisations, prescriptions, weight, vet visits, intake, maintenance, projects	RLS only (vet scope checked in code for vet visits); see CODE-1 and CODE-2
Upload routes (5)	<code>assertPhotoWriteAccess</code> before any Drive call; <code>record_attachment</code> re-checks in the database
Photo proxy GET	<code>is_public_drive_file</code> for public files; caller's RLS for private files
Status alerts POST, release mail POST	Bearer token equal to the service-role key, constant-time compare
Auth callback, manual PDF	Session required by proxy; <code>safeNextPath</code> (WEB-1); manual PDF has no own check (CODE-10)

6. Front end and authentication

Sign-in is email and password or Google, with temporary passwords for admin-created accounts, a forced change on first use, and authenticator-app two-step verification for the Settings → Security page only. Sessions last until sign-out. The public website reads only the curated public views.

The front end renders no raw HTML from users, validates every outbound link, and uses password-manager-friendly form attributes. The findings are an open-redirect bypass in the sign-in return path, the complete absence of HTTP security headers, an unbounded stranger sign-up path that can crowd real users off the Security page, and the fact that two-step verification is optional until an admin first opens Security.

6.1 What is done well

- **Role re-checked on every request.** The proxy loads the role per request, signs out archived or role-less sessions, and carries refreshed cookies on redirects (`src/lib/supabase/proxy.ts:110-141`). Archiving a user takes effect on their very next tap.
- **The must-change-password flag lives in `app_metadata`**, which the user cannot edit, and is cleared only after the password is actually replaced.
- **Two-step is verified with the auth server, not by decoding the cookie.** `getAuthenticatorAssuranceLevel` calls `getUser` first (`src/lib/auth/two-step.ts:40-42`); a forged `aal2` cookie was tested and rejected. All eight security actions enforce it server-side, not just the page.
- **Temporary passwords are strong and shown once.** Twelve characters from a no-look-alike alphabet via `crypto.randomInt`, returned once in the action result and never stored.
- **Forgot-password does not reveal whether an email exists** (`src/app/login/forgot/actions.ts:31-34`).
- **An admin cannot lock themselves out by accident:** own role, archive, delete and password reset are refused; create-user rolls back the auth row if the role insert fails.
- **No HTML injection surface.** The single `dangerouslySetInnerHTML` is JSON-LD with `<` escaped; site page bodies are parsed into React elements; names and notes are rendered as text.
- **Locked (pre-launch) mode leaks nothing:** the landing page reads no data, every response carries `X-Robots-Tag: noindex`, and `robots.txt` disallows all.
- **Password managers and authenticator apps work on phones:** `autoComplete="current-password"`, `"new-password"`, `"one-time-code"` and `inputMode="numeric"` are set on the right fields.
- **Destructive record flows have undo where it matters:** recording a death has an explicit undo page; archiving a user is reversible; deleting a user is refused when history exists.

6.2 Findings and mitigations

WEB-1 Open redirect after sign-in through a backslash in the return path Medium

What we found. The `next` parameter that returns a user to the page they were heading for is accepted if it starts with `/` and not `///`. Browsers and the URL parser treat `\` like `/` and strip tab and newline characters, so `next=//\evil.com` passes the check and resolves to `https://evil.com/`. Verified in this review with Node's URL parser against the exact code path.

Evidence.

- `src/lib/auth/next-path.ts:12` — `if (!next || !next.startsWith("/") || next.startsWith("//")) return null;`
- `src/lib/supabase/proxy.ts:73-75` — `NextResponse.redirect(new URL(signedInLandingPath(role, next), origin));` `src/app/login/actions.ts:40` — `redirect(...)` with the same value.
- Test: `new URL("/\\evil.com", "https://lannacare.org").href === "https://evil.com/"`.

Why it matters. A LINE or email message with a genuine-looking `https://lannacare.org/login?next=...` link lands a staff member on an attacker's look-alike page immediately after a real sign-in, which is the most convincing moment to ask them to “sign in again”. Requires the victim to click; does not by itself expose data.

Suggested mitigation.

- In `safeNextPath`: parse with `new URL(next, "http://x")`, return null unless `origin === "http://x"`, reject any backslash or control character, and return `pathname + search` rather than the raw string.

Effect on field users. None; genuine return paths still work.

Effort: Trivial (five lines).

WEB-2 No HTTP security headers, and session cookies are not marked Secure Medium

What we found. Nothing in `next.config.ts`, the proxy, the Worker or `wrangler.jsonc` sets `Content-Security-Policy`, `X-Frame-Options`, `Strict-Transport-Security`, `Referrer-Policy`, `Permissions-Policy` or `X-Content-Type-Options`. The Supabase session cookie uses the library defaults: `SameSite=Lax`, `HttpOnly=false` (required by the browser client), 400-day max age, and no `Secure` attribute is passed. Cloudflare may add HSTS at the zone level, which cannot be seen from the code.

Evidence.

- Grep for each header name across `src`, `worker`, `next.config.ts`, `wrangler.jsonc`: no matches.
- `node_modules/@supabase/ssr/dist/main/utils/constants.js:4-10` defaults; no `cookieOptions` passed in `src/lib/supabase/*.ts`.

Why it matters. The app can be framed by another site (clickjacking of admin pages); any future cross-site scripting bug would have the whole session cookie to steal, because it is readable by JavaScript; resident URLs leak to Google Maps and Drive through the referrer.

Suggested mitigation.

- Add `async headers()` in `next.config.ts` (or set them in `worker/index.mjs`): `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: strict-origin-when-cross-origin`, `Permissions-Policy: camera=(self), geolocation=()`, and `Strict-Transport-Security: max-age=31536000; includeSubDomains` if the zone does not already send it.
- Introduce a `Content-Security-Policy` in report-only mode first (`default-src 'self'; img-src 'self' data: blob;; frame-src https://www.google.com`) and tighten once the report is quiet.
- Pass `cookieOptions: { secure: true }` in production to both Supabase client factories.

Effect on field users. None. The camera permission stays allowed for photo capture and QR scanning.

Effort: Small (an hour, plus a week of CSP report-only observation).

WEB-3 Anyone can create an account by pressing “Continue with Google”, and the Security page reads only the first 200 users Medium

What we found. “Request access” is simply a Google sign-in; the callback refuses the session but the `auth.users` row stays and appears under Access requests. Because the anonymous key is public by design, an attacker can also call the Supabase sign-up API directly. The Security page and the access-request loader call `listUsers` with `perPage: 200` and never fetch page two.

Evidence.

- `src/app/login/request/page.tsx:12-17`; `src/app/auth/callback/route.ts:36-40`.
- `src/lib/auth/access-requests.ts:43` and `src/app/admin/security/page.tsx:22` — `listUsers({ perPage: LIST_USERS_PER_PAGE })`, page 1 only.

Why it matters. A flood of junk sign-ups pushes real staff logins off the users table (and out of the two-step and role lookups), so an admin cannot see or manage them. It is also an inbox-style spam surface for admins.

Suggested mitigation.

- Loop `listUsers` over pages (or filter server-side to users with roles plus the newest N requests).
- In Supabase Auth settings: turn off “Allow new users to sign up” (admin-created accounts still work through the service role) or restrict Google sign-in to the shelter's own domain; enable the built-in CAPTCHA only if abuse actually appears.
- Show requests newest-first with a cap and a “dismiss” that deletes the orphan auth row.

Effect on field users. Turning off open sign-up means a new volunteer must be invited by an admin rather than self-requesting. That matches how the shelter already works (temporary password handed over on LINE) and removes a confusing “you don't have access” page for newcomers.

Effort: Small.

WEB-4 Two-step verification is optional until an admin first opens Security Medium

What we found. Enrolment happens lazily on the first visit to Settings → Security. An admin who has never opened that page has no second factor at all, and any password-only session for that admin can enrol an authenticator of its own.

Evidence.

- `src/app/admin/security/verify/actions.ts:44-48` — starting enrolment requires only `hasAdminRole()`.
- `src/app/admin/security/UsersTable.tsx:32` — the table already knows which admins have `twoStep: false`.

Why it matters. A phished or guessed admin password is enough to gain full control of who has access, until that admin happens to enrol. Two-step protects only admins who have already set it up.

Suggested mitigation.

- Prompt enrolment at the first admin sign-in (a banner on My tasks that links to the setup page) and have the users table highlight admins without two-step.
- Optionally refuse the `admin` role assignment until the person has enrolled, or auto-open setup after the temporary-password change for admins.

Effect on field users. One-time setup per admin, already designed for phones (QR code, one code per sign-in). No effect on staff, vets or volunteers.

Effort: Small.

WEB-5 Password rule is eight characters with no leaked-password check Low

What we found. Minimum length is 8, enforced on client and server; there is no other rule. Supabase's "leaked password protection" (checks against known breach lists) is a dashboard setting and could not be verified from the code.

Evidence.

- `src/lib/auth/password-change.ts:47 MIN_PASSWORD_LENGTH = 8.`

Why it matters. Accounts that edit medical records can carry a weak or already-breached password.

Suggested mitigation.

- Raise the minimum to 10 or 12 and turn on leaked-password protection in Supabase. Do not add complexity rules (mixed case, symbols): they hurt phone typing more than they help and encourage reuse.

Effect on field users. A longer passphrase is easier to type on a phone than a short symbol-laden one; the password manager attributes already in place do the rest.

Effort: Trivial.

WEB-6 The contact map link on the public site is not scheme-checked Low

What we found. `contact_map_url` is saved with `optional()` and no `https` check (the form's `type="url"` is client-side only), then rendered as an `href` in the public footer and on adoption profiles. The other social links are validated in code and by a database constraint.

Evidence.

- `src/app/admin/website/actions.ts:144`; rendered at `PublicFooter.tsx:71` and `adopt/[id]/page.tsx:321`.

Why it matters. Only an admin can set it, so this is self-inflicted, but a `javascript:` value would run for every public visitor.

Suggested mitigation.

- Run it through the existing `checkHttpsUrl`, and add the same database check constraint the Facebook and Instagram links have.

Effect on field users. None.

Effort: Trivial.

WEB-7 Upload route handlers have no cross-site request check of their own Low

What we found. The five multipart upload routes are cookie-authenticated and have no Origin check or CSRF token. Server Actions get Next's built-in origin check; these handlers do not.

Evidence.

- `src/app/api/residents/[id]/photos/route.ts:31-45` and its four siblings.

Why it matters. Low in practice: the cookie is `SameSite=Lax`, so a cross-site form post from another origin does not carry it in modern browsers. Defence in depth only.

Suggested mitigation.

- Refuse when `Sec-Fetch-Site` is `cross-site` or when `Origin` is present and does not match the site origin (one shared helper, one line per route).

Effect on field users. None.

Effort: Trivial.

WEB-8 The proxy matcher skips any URL ending in an image extension, and public prefixes match loosely Low

What we found. The proxy is not run for paths ending in `.png`, `.svg`, `.jpg` and similar, which was intended for static assets but also covers dynamic routes such as `/residents/<id>.png`. Separately, public prefixes use `startsWith`, so `/adopt` also makes `/adoptXYZ` public.

Evidence.

- `src/proxy.ts:19` `matcher`; `src/lib/public-paths.ts` `PUBLIC_PATH_PREFIXES.some((p) => pathname.startsWith(p))` versus the segment-aware check in `isPublicPage`.

Why it matters. None today: page routes with a dot suffix 404 and RLS protects data. A future route handler with a dot-suffixed parameter, or a new top-level route beginning with a public word, would be unauthenticated by accident.

Suggested mitigation.

- Match only `/_next/static`, `/_next/image` and `/favicon.ico` in the exclusion; use segment-boundary matching in `isPublicPath` as `isPublicPage` already does.

Effect on field users. None.

Effort: Trivial.

WEB-9 OAuth and password-reset links are built from the forwarded Host header Low

What we found. `getSiteOrigin()` prefers `x-forwarded-host` when building the Google `redirectTo` and the password-reset link. A spoofed header at the origin would put an attacker's host in those links; Supabase's redirect allow-list should bounce it, and the Pi origin is protected by a WAF key.

Evidence.

- `src/lib/site-origin.ts:17-21`; used at `src/app/login/actions.ts:53` and `src/app/login/forgot/actions.ts:29`.

Why it matters. Low; several layers would have to fail together.

Suggested mitigation.

- Set `NEXT_PUBLIC_SITE_URL` per environment so the header is never consulted (the code already prefers it).

Effect on field users. None.

Effort: Trivial.

WEB-10 Destructive actions mostly use the browser's built-in confirm box, which is easy to tap through on a phone Low

What we found. A purpose-built `ConfirmDialog` exists (cancel focused, consequences listed, large buttons) but is used in only six files. About twenty-five other files use `window.confirm`, including archive, delete and two-step reset on the Security page. Admin table buttons are small (`text-xs px-2 py-1`). There is no offline notice: a form submitted with no signal shows a generic framework error.

Evidence.

- Grep of `src` for `window.confirm` / `confirm`: 25 files; `src/app/admin/security/UsersTable.tsx:114-145,222`; no `navigator.onLine` handling anywhere.

Why it matters. This is the human side of security: a reflex “OK” on a tiny native dialog in a kennel deletes a record that (DB-6) cannot be recovered. Offline failures look like bugs and lead to double submissions.

Suggested mitigation.

- Migrate the Security page and every record-deletion confirm to `ConfirmDialog`; name the thing being deleted in the dialog.
- Give admin table buttons the same 44 px minimum height the public header already uses.
- Add a small “You're offline, changes will not save” banner driven by `navigator.onLine` and a failed-fetch fallback.

Effect on field users. Directly improves it.

Effort: Medium (a few hours across files).

WEB-11 Sessions never expire (a deliberate usability choice) Info

What we found. The auth project is configured with no session time-box and no inactivity timeout; the cookie lives 400 days. A phone that is signed in stays signed in until someone signs out or an admin archives the login.

Evidence.

- `docs/decisions/2026-09-27-settings-security-requires-2-step-verification-the-feature.md` records `jwt_exp 3600`, no timebox, no inactivity timeout (dev project).

Why it matters. Right for people working in kennels who must not be asked to re-type a password with wet hands. The residual risk is a lost or sold phone that stays signed in, which the archive button covers as long as someone reports it.

Suggested mitigation.

- Keep no time-box. Set an inactivity timeout of 60 days in Supabase: a phone used weekly never sees it; a lost phone does.
- Write a one-line “lost phone” procedure in the manual: tell an admin, who archives and restores the login (which ends every session).

Effect on field users. No change for active users.

Effort: Trivial (dashboard setting plus a manual line).

WEB-12 No two-step recovery codes; recovery relies on a second admin or a developer script[Info](#)

What we found. This is a documented decision. Recovery is another admin pressing Reset, or `scripts/bootstrap-admin.mjs --reset-2step` run by the developer with the service role.

Evidence.

- README, Roles; the decision record of 2026-09-27.

Why it matters. If the only enrolled admin loses their phone while the developer is unavailable, Settings → Security is unreachable until they are.

Suggested mitigation.

- Keep the documented rule of at least two enrolled admins and check it on the users table (WEB-4 makes that visible). Revisit recovery codes if Supabase promotes the API out of experimental.

Effect on field users. None.

Effort: None now.

7. Prioritised action plan

Effort is the developer's time; “Shelter” marks decisions the shelter's leadership should make rather than the developer. The first phase removes the High finding and the two most exposed front-end weaknesses in about a day. Everything in the first two phases can ship as ordinary feature PRs under the repository's existing test-plan rules.

Phase 1 — this week (about one day)

Ref	Action	Owner	Effort
DB-1	Encrypt backups; exclude auth session and MFA tables; move to an admin-only Drive folder; stop logging the link.	Developer	Half day
WEB-1	Harden <code>safeNextPath</code> (origin comparison, reject backslash and control characters).	Developer	Trivial
WEB-2	Add security headers; CSP report-only; <code>secure</code> cookie flag in production.	Developer	1 hour
CODE-1/2	Check row counts before Drive deletes; delete the job before its attachments.	Developer	1 hour
CODE-4/6/7	CSV formula prefix; Drive backslash escaping; strict PDF signature; nosniff on the proxy.	Developer	1 hour
WEB-5/11	Supabase dashboard: min password 10-12, leaked-password check on, 60-day inactivity timeout, confirm TOTP enabled on production, confirm rate limits.	Developer	30 min

Phase 2 — this month

Ref	Action	Owner	Effort
DB-2	Move the backup to an always-on runner; rehearse and document a restore.	Developer	1 day
DB-3/4/9	Migrations: guard <code>end_date</code> ; refuse untyped placements into Lifecycle enclosures; protect Lifecycle names.	Developer	Half day
WEB-3	Paginate <code>listUsers</code> ; turn off open sign-up or restrict Google domain; add “dismiss request”.	Developer + Shelter	Half day
WEB-4	Prompt admin two-step enrolment at first sign-in; flag un-enrolled admins in the users table.	Developer	Half day
DB-5/8	Narrow vet (and volunteer) read of contacts and staff emails.	Shelter decides, Developer implements	Half day
CODE-3	Move PAT and DB password out of plain env files; scoped tokens; rotate.	Developer	Half day
CODE-5	Map remaining database errors to plain sentences.	Developer	Half day
CODE-8/9	Cloudflare rate-limit rule on <code>/api/*</code> and <code>/login</code> ; bump wrangler; audit step in CI; pin actions.	Developer	1 hour

Ref	Action	Owner	Effort
WEB-6/7/8/9	Validate map URL; cross-site check on upload routes; tighten proxy matcher and prefix matching; set <code>NEXT_PUBLIC_SITE_URL</code> per environment.	Developer	2 hours

Phase 3 — next quarter

Ref	Action	Owner	Effort
DB-6	Audit-log trigger on record tables; prefer archive over delete for medical rows; optional “recent changes” page.	Developer	1-2 days
DB-7	Decide whether the full roster stays public; if not, gate cards and add random suffixes to new codes.	Shelter decides, Developer implements	Half day
WEB-10	Replace <code>window.confirm</code> with <code>ConfirmDialog</code> ; larger admin tap targets; offline banner.	Developer	1 day
DB-10	Extend the static migration checker; run the public-view check in CI; views over the <code>site_*</code> tables.	Developer	Half day
DB-11	Retention period for assistant transcripts.	Shelter decides	Trivial
CODE-10	Session check in the manual PDF route; shared layout guard for app pages.	Developer	1 hour
—	Re-run this assessment's checks after the changes; consider a short external penetration test before the public launch.	Developer + Shelter	—

8. Appendix

A. Settings to verify outside the code

These live in dashboards and could not be checked from the repository. Each is a five-minute check.

Where	Setting	Recommended
Supabase Auth	Allow new users to sign up	Off (admin-created accounts still work), or Google restricted to the shelter's domain
Supabase Auth	Minimum password length; leaked password protection	10-12; On
Supabase Auth	Session inactivity timeout; time-box	60 days; none
Supabase Auth	MFA (TOTP) enroll and verify enabled on production	On (the decision record flags this as unchecked)
Supabase Auth	Rate limits: sign-in, OTP, MFA verify, email	Defaults or tighter; confirm MFA verify limit exists
Supabase Auth	Redirect URL allow-list	Only the real site origins and <code>localhost</code> for dev
Cloudflare	HSTS at zone level; Always use HTTPS	On, max-age one year
Cloudflare	Rate limiting rule on <code>/api/*</code> and <code>/login</code>	One rule, for example 100 requests per minute per IP
Cloudflare	WAF managed rules; bot fight mode	On, free tier
Google Drive	Sharing of the app root and Backups folders	Backups: administrator only. Root: only the app's account and the administrator
Google Cloud	OAuth app publishing status	Published (Testing-mode refresh tokens expire after seven days)
Laptop	Disk encryption (BitLocker); OS account password; screen lock	On; strong; short

B. How each finding was verified

- Every file and line reference was opened and read by the lead reviewer after the area reviews reported it; claims that did not match the code were dropped.
- The open redirect (WEB-1) was reproduced with Node 24's WHATWG URL parser: a `next` value of a slash followed by two backslashes and a host resolves to that host when combined with the site origin.
- Security headers (WEB-2) were confirmed absent by searching for each header name across the source, Worker and configuration files.
- Committed secrets were ruled out by listing every file ever added in git history and searching all history for key material.
- Dependency advisories (CODE-9) come from `npm audit --omit=dev` run on 30 September 2026.
- The unguarded-page list (CODE-10) is from a search of every `page.tsx` for the guard helpers, excluding the public pages.

C. Verification commands for after remediation

- `node scripts/check-public-views.mjs` — proves the anonymous role is refused everywhere but the allow-list (run against test after DB-5, DB-7, DB-8, DB-10).
- `node -e "console.log(new URL('/\\\\\\\\evil.com', 'https://lannacare.org').href)"` — after WEB-1, `safeNextPath` must return null for this input.
- `curl -sI https://test.lannacare.org/login | grep -iE 'strict-transport|x-frame|content-security|referrer-policy|x-content-type'` — after WEB-2, all five headers present.
- `npm audit --omit=dev` — after CODE-9, zero high.
- `pg_restore --list <dump>` on an encrypted-then-decrypted backup — after DB-1, no `auth.refresh_tokens`, `auth.sessions` or `auth.mfa_*` entries.

D. Finding index

Ref	Severity	Title
DB-1	High	Database backups are unencrypted and include sign-in secrets
DB-2	Medium	Backups depend on one laptop being awake, and a restore has never been rehearsed
DB-3	Medium	Placement history is not truly append-only: staff can rewrite end dates
DB-4	Medium	A plain enclosure move into a Lifecycle pseudo-enclosure can silently mark an animal deceased or adopted and lock its record
DB-5	Medium	Outside vet clinics and volunteers can read the whole contacts address book
DB-6	Medium	No general audit trail for edits and deletes of resident, medical and contact records
DB-7	Medium	The public resident card and enclosure views publish the whole roster, and the card codes are guessable
DB-8	Low	Every staff member's email is visible to vets and volunteers
DB-9	Low	Staff can rename or delete the Lifecycle zone and its pseudo-enclosures that the status logic keys on
DB-10	Low	Static migration checks do not cover RLS, <code>search_path</code> or anon revokes on new objects
DB-11	Info	Assistant transcripts are kept verbatim, forever
CODE-1	Medium	A Drive file is deleted even when the database quietly refused the delete
CODE-2	Medium	Deleting a maintenance job removes its attachment records before finding out the caller may not delete the job
CODE-3	Medium	All production secrets sit in plain-text files on one laptop and are copied into every worktree
CODE-4	Low	CSV exports do not neutralise spreadsheet formulas
CODE-5	Low	Raw database error text is shown to users in several forms
CODE-6	Low	Drive folder-name queries escape quotes but not backslashes
CODE-7	Low	PDF signature check is loose and the photo proxy sets no content-type protection
CODE-8	Low	No application-level rate limiting; a signed-in user could burn the Drive quota
CODE-9	Low	Dependency advisories in development tooling; no audit step in CI; actions not pinned

Ref	Severity	Title
CODE-10	Info	About twenty pages and the manual PDF route rely on the proxy alone for the sign-in redirect
WEB-1	Medium	Open redirect after sign-in through a backslash in the return path
WEB-2	Medium	No HTTP security headers, and session cookies are not marked Secure
WEB-3	Medium	Anyone can create an account by pressing “Continue with Google”, and the Security page reads only the first 200 users
WEB-4	Medium	Two-step verification is optional until an admin first opens Security
WEB-5	Low	Password rule is eight characters with no leaked-password check
WEB-6	Low	The contact map link on the public site is not scheme-checked
WEB-7	Low	Upload route handlers have no cross-site request check of their own
WEB-8	Low	The proxy matcher skips any URL ending in an image extension, and public prefixes match loosely
WEB-9	Low	OAuth and password-reset links are built from the forwarded Host header
WEB-10	Low	Destructive actions mostly use the browser's built-in confirm box, which is easy to tap through on a phone
WEB-11	Info	Sessions never expire (a deliberate usability choice)
WEB-12	Info	No two-step recovery codes; recovery relies on a second admin or a developer script